Group No: 04

Group Members

SL	Name	Student ID (Roll)
1	Md. Masud Rana	2211176104
2	Md. Shibli Noman Sarkar	2210776130
3	Md. Majnu Miah	2210776138
4	Md. Ruhul Amin	2210776147
5	Md. Naimur Rahman	2111077144

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit

Importing Required Libraries

This cell imports all the necessary libraries required to build, train, and evaluate the Deep Neural Network (DNN).

Libraries Used

- **TensorFlow (`tensorflow`)**
  - Provides the deep learning framework used to build and train the neural network.
- **NumPy (`numpy`)**
  - Used for numerical computations and handling arrays.
- **Matplotlib (`matplotlib.pyplot`)**
  - Used to visualize the training and validation performance of the model through graphs.
- **Scikit-learn (`train_test_split`)**
  - Splits the dataset into training, validation, and testing subsets.

- **Scikit-learn (`MinMaxScaler`)**
  - Normalizes the input and output data to the range **[-1, 1]**, which improves the stability and convergence of the neural network during training.
- **TensorFlow Keras Layers**
  - `Input`: Defines the input layer of the neural network.
  - `Dense`: Creates fully connected (dense) layers.
  - `Activation`: Applies activation functions to the network layers.
- **TensorFlow Keras Model**
  - `Model`: Used to construct the Deep Neural Network using the Functional API.
- **TensorFlow Keras Metrics**
  - Provides evaluation metrics to monitor the model's performance during training.
- **MeanSquaredError**
  - Defines the Mean Squared Error (MSE) loss function, which is appropriate for regression problems where the objective is to predict continuous values.

```
# importing necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.layers import Dense, Input, Activatio
from tensorflow.keras.models import Model
from tensorflow.keras import metrics
from tensorflow.keras.losses import MeanSquaredError
```

```
2026-08-05 22:49:36.444537: I tensorflow/core/platform/cpu_feature_guard.
To enable the following instructions: SSE4.1 SSE4.2 AVX AVX2 FMA, in othe
```

Defining the Polynomial Function

This cell defines the polynomial function that will be approximated by the Deep Neural Network (DNN).

The mathematical expression of the function is:

[y = 5x^3 - 10x^2 - 20x + 10]

The function takes an input value `x` and returns the corresponding output `y` according to the polynomial equation.

Function Definition

```
def polynomial(x):
    return 5 * x**3 - 10 * x**2 - 20 * x + 10
```

Purpose

- Implements the target polynomial equation.
- Generates the output values (`y`) for the corresponding input values (`x`).
- Creates the dataset that will be used to train, validate, and test the Deep Neural Network.

```
1 # Polynomial function
2 def polynomial(x):
3     return 5 * x**3 - 10 * x**2 - 20 * x + 10
```

Generating the Dataset

This cell generates the input (`x`) and output (`y`) data that will be used to train the Deep Neural Network (DNN).

Dataset Generation

```
# Generate input values
x = np.linspace(-20, 20, 5000).reshape(-1,1)

# Compute corresponding output values
y = polynomial(x)
```

Explanation

- `np.linspace(-20, 20, 5000)` generates **5,000 evenly spaced input values** between **-20** and **20**.

- `.reshape(-1,1)` converts the input array into a **column vector** with shape `(5000, 1)`, which is the required format for training the neural network.
- `polynomial(x)` computes the corresponding output values using the polynomial equation:

[y = 5x^3 - 10x^2 - 20x + 10]

Purpose

- Generate input samples within the required range of **-20 to +20**.
- Calculate the target output values for each input sample.
- Create the complete dataset that will be normalized and split into training, validation, and testing sets in the subsequent steps.

```
# Generate data
x = np.linspace(-20, 20, 5000).reshape(-1,1)
y = polynomial(x)
```

Data Normalization

This cell normalizes both the input (`x`) and output (`y`) data to the range **[-1, 1]** using the **MinMaxScaler** from Scikit-learn.

Code

```
# Normalize to [-1, 1]
x_scaler = MinMaxScaler(feature_range=(-1,1))
y_scaler = MinMaxScaler(feature_range=(-1,1))

x_norm = x_scaler.fit_transform(x)
y_norm = y_scaler.fit_transform(y)
```

Explanation

- `MinMaxScaler(feature_range=(-1,1))` creates a scaler that transforms the data to the range **-1 to +1**.
- `fit_transform()` first learns the minimum and maximum values of the dataset and then scales every value into the specified range.
- Separate scalers are used for the input (`x`) and output (`y`) because they have different value ranges.

Why Normalize the Data?

Neural networks generally perform better when the input and output values are on a similar scale. Normalizing the data:

- Scales all values to the range **[-1, 1]**.
- Speeds up model convergence during training.
- Improves numerical stability.
- Prevents features with large values from dominating the learning process.
- Helps the optimizer learn more efficiently.

Output

After normalization:

- `x_norm` contains the normalized input values in the range **[-1, 1]**.
- `y_norm` contains the normalized target values in the range **[-1, 1]**, ready for training the Deep Neural Network.

```
# Normalize to [-1, 1]
x_scaler = MinMaxScaler(feature_range=(-1,1))
y_scaler = MinMaxScaler(feature_range=(-1,1))

x_norm = x_scaler.fit_transform(x)
y_norm = y_scaler.fit_transform(y)
```

Visualizing the Polynomial Function

This cell plots the polynomial function to visualize the relationship between the input variable (`x`) and the corresponding output (`y`).

Code

```
# Simple plot of the polynomial function
plt.figure(figsize=(8,5))
plt.plot(x, y, color='blue', linewidth=2)
plt.title("Polynomial Function")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Explanation

- `plt.figure(figsize=(8,5))` creates a figure with a size of **8 × 5 inches**.
- `plt.plot(x, y, color='blue', linewidth=2)` plots the polynomial curve using a blue line with a thickness of **2**.
- `plt.title("Polynomial Function")` adds a title to the graph.
- `plt.xlabel("x")` labels the horizontal axis.
- `plt.ylabel("y")` labels the vertical axis.
- `plt.show()` displays the plot.

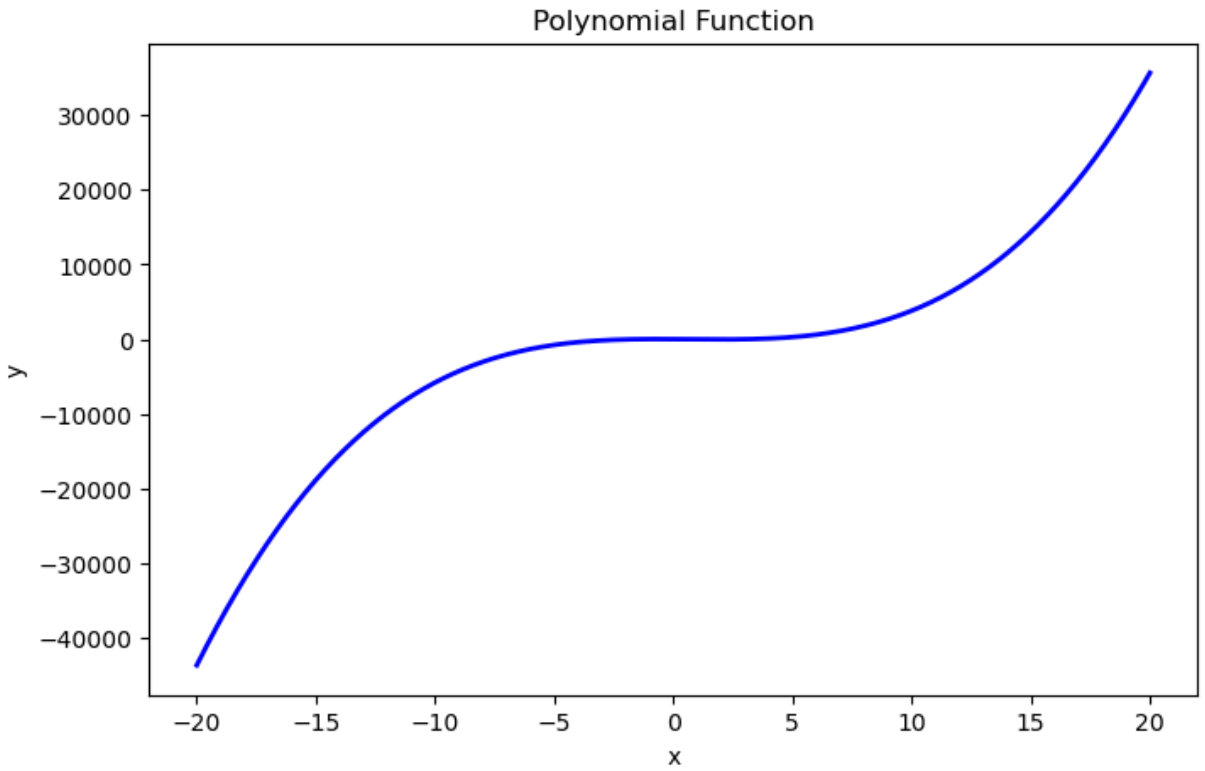
Purpose

- Visualize the shape and behavior of the polynomial function before training the Deep Neural Network.
- Verify that the generated dataset correctly represents the equation

[y = 5x^3 - 10x^2 - 20x + 10.]

- Gain insight into how the output varies with the input over the range **-20 to +20**.

```
# simple plot this polinomial value
plt.figure(figsize=(8,5))
plt.plot(x, y, color='blue', linewidth=2)
plt.title("Polynomial Function")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



Visualizing the Normalized Polynomial Function

This cell plots the **normalized input (`x_norm`)** against the **normalized output (`y_norm`)** to verify that the normalization process has scaled the data to the required range **[-1, 1]** while preserving the overall shape of the polynomial function.

Code

```
plt.figure(figsize=(8,5))
plt.plot(x_norm, y_norm, color='blue', linewidth=2)
plt.title("Normalized Polynomial Function")
plt.xlabel("Normalized x")
plt.ylabel("Normalized y")
plt.show()
```

Explanation

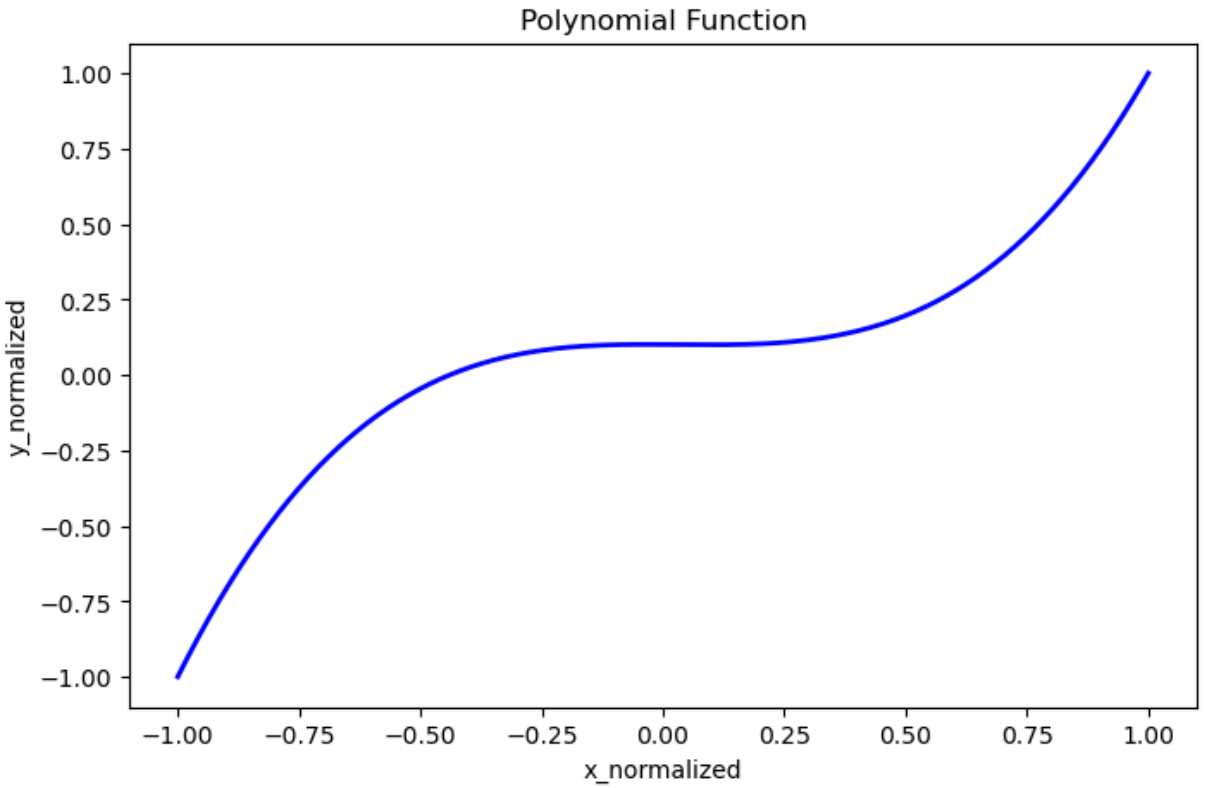
- `plt.figure(figsize=(8,5))` creates a figure with dimensions **8 × 5 inches**.

- `plt.plot(x_norm, y_norm, color='blue', linewidth=2)` plots the normalized polynomial curve using a blue line with a width of **2**.
- `plt.title("Normalized Polynomial Function")` adds a descriptive title to the graph.
- `plt.xlabel("Normalized x")` labels the horizontal axis with the normalized input values.
- `plt.ylabel("Normalized y")` labels the vertical axis with the normalized output values.
- `plt.show()` displays the graph.

Purpose

- Verify that both the input and output data have been successfully normalized to the range **[-1, 1]**.
- Confirm that normalization preserves the original polynomial relationship while changing only the scale of the data.
- Ensure the dataset is properly prepared for efficient training of the Deep Neural Network (DNN).

```
plt.figure(figsize=(8,5))
plt.plot(x_norm, y_norm, color='blue', linewidth=2)
plt.title("Polynomial Function")
plt.xlabel("x_normalized")
plt.ylabel("y_normalized")
plt.show()
```



Splitting the Dataset

This cell divides the normalized dataset into **training** and **temporary** datasets. The temporary dataset will later be split equally into **validation** and **testing** datasets.

Code

```
# Splitting the data into training and temporary sets
X_train, X_temp, y_train, y_temp = train_test_split(
    x_norm,
    y_norm,
    test_size=0.20,
    random_state=42
)
```

Explanation

- `train_test_split()` randomly partitions the normalized input (`x_norm`) and output (`y_norm`) data.
- `test_size=0.20` reserves **20%** of the dataset as a temporary set (`X_temp`, `y_temp`), while the remaining **80%** is used for training.
- `random_state=42` ensures that the data split is reproducible, meaning the same split will be obtained each time the code is executed.

Dataset Distribution

After this step:

Dataset	Percentage
Training Set	80%
Temporary Set	20%

The temporary dataset will be divided equally in the next step to create:

- **10% Validation Data**
- **10% Test Data**

Purpose

- Create a training dataset for learning the polynomial relationship.
- Reserve a temporary dataset that will later be split into validation and testing sets.
- Ensure a reproducible and unbiased random data split for model evaluation.

```
# splitting the data into training and testing sets
X_train, X_temp, y_train, y_temp = train_test_split(
    x_norm,
    y_norm,
    test_size=0.20,
    random_state=42
)
```

```
X_train.shape
```

```
(4000, 1)
```

Creating the Validation and Test Sets

This cell further divides the temporary dataset into **validation** and **test** datasets. Since the temporary dataset contains **20%** of the original data, splitting it equally results in **10% validation data** and **10% test data**.

Code

```
X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42
)
```

Explanation

- `train_test_split()` is applied to the temporary dataset (`X_temp` and `y_temp`).
- `test_size=0.50` divides the temporary dataset into two equal parts:
  - **50%** for the validation set.
  - **50%** for the test set.
- `random_state=42` ensures that the split is reproducible, producing the same validation and test sets each time the code is executed.

Final Dataset Distribution

After completing both splitting steps, the dataset is divided as follows:

Dataset	Percentage
Training Set	80%
Validation Set	10%
Test Set	10%

Purpose

- **Training Set:** Used to train the Deep Neural Network by updating its weights.
- **Validation Set:** Used during training to monitor the model's performance, tune hyperparameters, and detect overfitting.
- **Test Set:** Used after training to evaluate the model's final performance on previously unseen data.

```
1 X_val, X_test, y_val, y_test = train_test_split(
2     X_temp,
3     y_temp,
4     test_size=0.50,
```

```
    random_state=42
)
```

Building the Deep Neural Network (DNN)

This cell defines the architecture of the **Deep Neural Network (DNN)** using TensorFlow's **Sequential API**. The model consists of one input layer, three hidden layers, and one output layer for solving the polynomial regression problem.

Code

```
# Build model
model = tf.keras.Sequential([
    Input(shape=(1,)),
    Dense(32, activation='relu'),
    Dense(64, activation='relu'),
    Dense(128, activation='relu'),
    Dense(1)
])
```

Model Architecture

Layer	Number of Neurons	Activation Function
Input Layer	1	—
Hidden Layer 1	32	ReLU
Hidden Layer 2	64	ReLU
Hidden Layer 3	128	ReLU
Output Layer	1	Linear (default)

Explanation

- **Input Layer (`Input(shape=(1,))`)**
  - Accepts a single input feature (`x`) for each sample.
- **Hidden Layer 1**
  - Contains **32 neurons** with the **ReLU (Rectified Linear Unit)** activation function to learn nonlinear relationships.
- **Hidden Layer 2**

- Contains **64 neurons** with the **ReLU** activation function, allowing the network to learn more complex features.

- **Hidden Layer 3**
  - Contains **128 neurons** with the **ReLU** activation function, increasing the model's capacity to approximate the polynomial function.
- **Output Layer**
  - Contains **1 neuron** with the default **linear activation**, making it suitable for predicting a continuous numerical value in a regression task.

Purpose

- Construct a Deep Neural Network with **three hidden layers of sizes 32, 64, and 128**, as specified in the assignment.
- Learn the nonlinear mapping between the input variable (`x`) and the target output (`y`).
- Produce a continuous output corresponding to the predicted value of the polynomial function.

```
# Build model
model = tf.keras.Sequential([
    Input(shape=(1,)),
    Dense(32, activation='relu'),
    Dense(64, activation='relu'),
    Dense(128, activation='relu'),
    Dense(1)
])

```

Compiling the Deep Neural Network

This cell configures the Deep Neural Network (DNN) for training by specifying the optimizer, loss function, and evaluation metric.

Code

```
model.compile(
    optimizer='adam',
    loss='mse',
```

```
    metrics=['mae']
)
```

Explanation

- **Optimizer:** `adam`
  - The **Adam (Adaptive Moment Estimation)** optimizer is used to update the model's weights during training.
  - It combines the advantages of Momentum and RMSProp, providing faster convergence and efficient learning.
- **Loss Function:** `mse` **(Mean Squared Error)**
  - Measures the average squared difference between the predicted values and the actual target values.
  - It is the standard loss function for regression problems because it penalizes larger prediction errors more heavily.
- **Metric:** `mae` **(Mean Absolute Error)**
  - Computes the average absolute difference between the predicted and actual values.
  - MAE is easier to interpret than MSE since it represents the average prediction error in the same scale as the normalized target values.

Purpose

- Configure the model for training.
- Use the **Adam optimizer** to efficiently minimize the prediction error.
- Optimize the model using **Mean Squared Error (MSE)**.
- Monitor the model's performance during training using **Mean Absolute Error (MAE)**.

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

Training the Deep Neural Network

This cell trains the Deep Neural Network (DNN) using the training dataset while monitoring its performance on the validation dataset after each epoch.

Code

```
# Train the model
history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=32,
    verbose=1
)
```

Explanation

- **`X_train`**, **`y_train`**
  - The normalized training data used to train the neural network.
- **`validation_data=(X_val, y_val)`**
  - Evaluates the model on the validation dataset after each epoch.
  - Helps monitor the model's ability to generalize to unseen data.
- **`epochs=100`**
  - The model is trained for **100 complete passes (epochs)** through the training dataset.
  - During each epoch, the network updates its weights to minimize the loss function.
- **`batch_size=32`**
  - The training data is divided into mini-batches of **32 samples**.
  - Model parameters are updated after processing each batch, making training more efficient.
- **`verbose=1`**
  - Displays detailed training progress, including the training loss, validation loss, and evaluation metric for each epoch.

Output

The `fit()` function returns a **History** object stored in the variable `history`. It contains the values of:

- Training Loss (`loss`)
- Validation Loss (`val_loss`)
- Training Mean Absolute Error (`mae`)
- Validation Mean Absolute Error (`val_mae`)

These values are later used to visualize the learning process by plotting:

- **Training MAE vs. Validation MAE**
- **Training Loss vs. Validation Loss**

Purpose

- Train the Deep Neural Network to learn the polynomial relationship between the normalized input and output data.
- Monitor the model's performance on both the training and validation datasets during the learning process.
- Store the training history for performance analysis and visualization.

```
# Train
history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0190 - mae: 0.0900 -
Epoch 2/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 7.6188e-04 - mae: 0.01
Epoch 3/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.3096e-04 - mae: 0.00
Epoch 4/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.0050e-05 - mae: 0.00
Epoch 5/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.1463e-05 - mae: 0.00
Epoch 6/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.6739e-05 - mae: 0.00
Epoch 7/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.4383e-05 - mae: 0.00
Epoch 8/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 9.7871e-06 - mae: 0.00
Epoch 9/100
```

```
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.0072e-05 - mae: 0.00
Epoch 10/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 9.6209e-06 - mae: 0.00
Epoch 11/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.0264e-05 - mae: 0.00
Epoch 12/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.6234e-05 - mae: 0.00
Epoch 13/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.4838e-06 - mae: 0.00
Epoch 14/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 6.5051e-06 - mae: 0.00
Epoch 15/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 6.6077e-06 - mae: 0.00
Epoch 16/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.0007e-06 - mae: 0.00
Epoch 17/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 8.8791e-06 - mae: 0.00
Epoch 18/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.3487e-05 - mae: 0.00
Epoch 19/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 7.8295e-05 - mae: 0.00
Epoch 20/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.3833e-05 - mae: 0.00
Epoch 21/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 8.9034e-06 - mae: 0.00
Epoch 22/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.8951e-05 - mae: 0.00
Epoch 23/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.8079e-06 - mae: 0.00
Epoch 24/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 5.6120e-06 - mae: 0.00
Epoch 25/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 4.9142e-06 - mae: 0.00
Epoch 26/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.5989e-05 - mae: 0.00
Epoch 27/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2.2498e-04 - mae: 0.00
Epoch 28/100
125/125 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.1831e-05 - mae: 0.00
Epoch 29/100
```

Plotting the Training and Validation Loss

This cell visualizes the model's learning progress by plotting the **training loss** and **validation loss** over all training epochs. The loss values are obtained from the `history` object returned by the `model.fit()` function.

Code

```
plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.title('Training and Validation Loss Over Epochs')
plt.grid(True)
plt.show()
```

Explanation

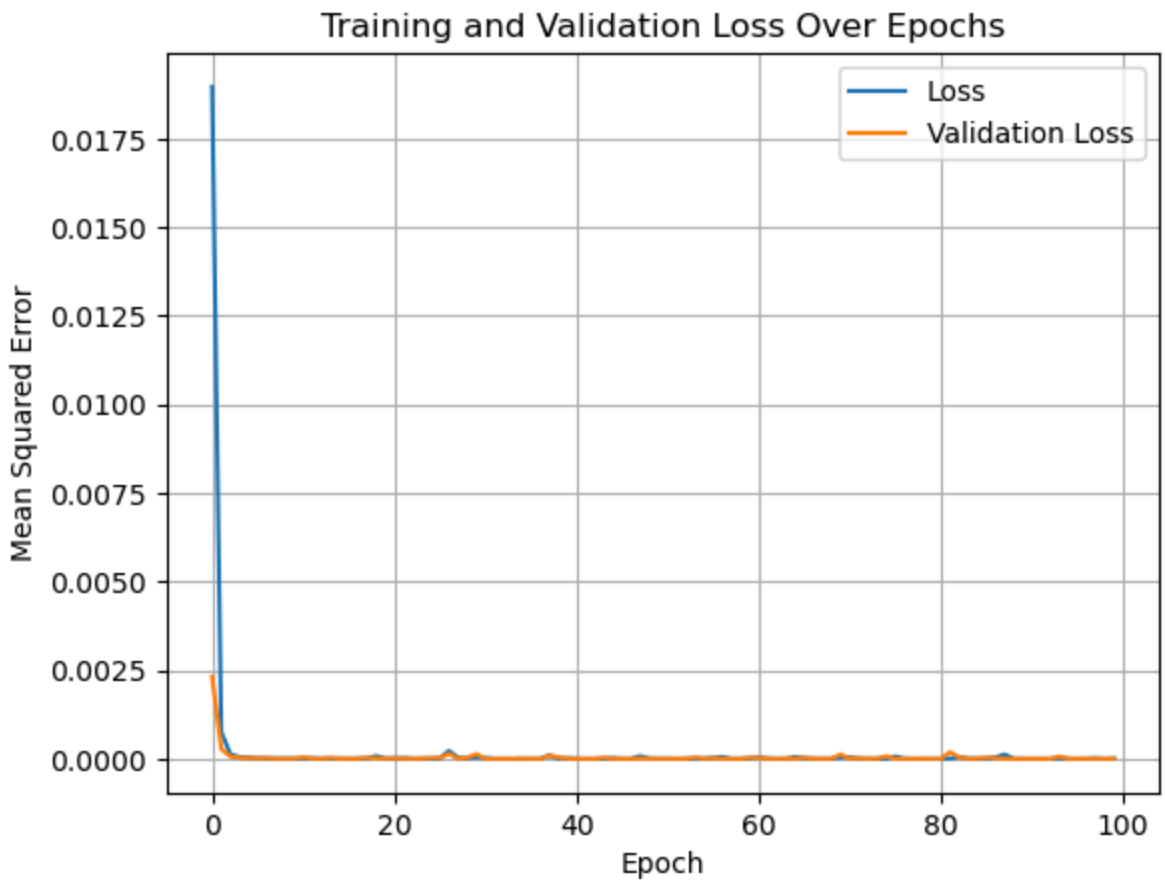
- `history.history['loss']`
  - Retrieves the **training loss (Mean Squared Error)** recorded after each epoch.
- `history.history['val_loss']`
  - Retrieves the **validation loss (Mean Squared Error)** recorded after each epoch.
- `plt.plot()`
  - Draws the training and validation loss curves.
- `plt.xlabel('Epoch')`
  - Labels the horizontal axis with the training epochs.
- `plt.ylabel('Mean Squared Error')`
  - Labels the vertical axis with the loss values (MSE).
- `plt.legend()`
  - Displays a legend to distinguish between the training and validation loss curves.
- `plt.title()`
  - Adds a descriptive title to the graph.
- `plt.grid(True)`
  - Displays grid lines for easier interpretation of the learning curves.
- `plt.show()`
  - Renders the plot.

Purpose

- Visualize how the model's error changes during training.

- Compare the training and validation loss at each epoch.
- Monitor the learning process and identify potential **underfitting** or **overfitting**.
- Evaluate whether the model is converging toward an optimal solution.

```
plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['val_loss'], label='Validation Los
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.title('Training and Validation Loss Over Epochs')
plt.grid(True)
plt.show()
```



Evaluating the Model on the Test Dataset

This cell evaluates the trained Deep Neural Network (DNN) using the **test dataset**, which contains data that was not used during training or validation. It reports the model's **Mean Squared Error (MSE)** and **Mean Absolute Error (MAE)** to measure its predictive performance.

Code

```
loss, mae = model.evaluate(X_test, y_test, verbose=0)

print("\nTest MSE :", loss)
print("Test MAE :", mae)
```

Explanation

- `model.evaluate(X_test, y_test)`
  - Evaluates the trained model using the test dataset.
  - Returns the values of the loss function and the evaluation metric specified during model compilation.
- `loss`
  - Stores the **Mean Squared Error (MSE)** on the test dataset.
  - A lower MSE indicates that the model's predictions are closer to the actual target values.
- `mae`
  - Stores the **Mean Absolute Error (MAE)** on the test dataset.
  - MAE represents the average absolute difference between the predicted and actual values.
- `verbose=0`
  - Suppresses the progress bar and displays only the final evaluation results.

Output

The following performance metrics are displayed:

- **Test MSE:** Measures the average squared prediction error.
- **Test MAE:** Measures the average absolute prediction error.

Purpose

- Evaluate the final performance of the trained Deep Neural Network on unseen data.
- Assess the model's ability to generalize beyond the training and validation datasets.
- Quantify the prediction error using the regression metrics **Mean Squared Error (MSE)** and **Mean Absolute Error (MAE)**.

```
loss, mae = model.evaluate(X_test, y_test, verbose=0)

print("\nTest MSE :", loss)
print("Test MAE :", mae)
```

```
Test MSE : 4.357575562607963e-06
Test MAE : 0.0016113981837406754
```

Calculating the R^2 (Coefficient of Determination) Score

This cell evaluates the regression model using the **R^2 (Coefficient of Determination)** score. The R^2 score measures how well the Deep Neural Network (DNN) explains the variability of the target data.

Code

```
from sklearn.metrics import r2_score

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
print(f'R2 Score on Test Set: {r2:.6f}')
```

Explanation

- `from sklearn.metrics import r2_score`
  - Imports the `r2_score` function from Scikit-learn, which is used to evaluate the goodness of fit of a regression model.
- `model.predict(X_test)`
  - Uses the trained Deep Neural Network to generate predictions for the test dataset.
  - The predicted values are stored in `y_pred`.
- `r2_score(y_test, y_pred)`
  - Computes the **R^2 score** by comparing the actual target values (`y_test`) with the predicted values (`y_pred`).
- `print(f'R2 Score on Test Set: {r2:.6f}')`
  - Displays the R^2 score with six decimal places.

Understanding the R^2 Score

The R^2 score ranges from **$-\infty$ to 1**:

R^2 Score	Interpretation
1.0	Perfect prediction; the model explains 100% of the variance.
0.0	The model performs no better than predicting the mean of the target values.
< 0	The model performs worse than simply predicting the mean.

A value **closer to 1** indicates that the model accurately captures the relationship between the input and output data.

Purpose

- Evaluate the regression model using the **Coefficient of Determination (R^2)**.
- Measure how well the trained DNN predicts unseen test data.
- Provide an additional performance metric alongside **Mean Squared Error (MSE)** and **Mean Absolute Error (MAE)** for a comprehensive assessment of the model.

```
from sklearn.metrics import r2_score

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
print(f'R2 Score on Test Set: {r2:.6f}')
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
R2 Score on Test Set: 0.999972
```

Plotting the Training and Validation Mean Absolute Error (MAE)

This cell visualizes the **Mean Absolute Error (MAE)** of the model on both the training and validation datasets over each training epoch. The MAE curves help assess how the prediction error changes during the learning process.

Code

```
# Plot Training vs Validation MAE

plt.figure(figsize=(8,5))
plt.plot(history.history['mae'], label='Training MAE')
plt.plot(history.history['val_mae'], label='Validation MAE')
```

```
plt.title("Training MAE vs Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.legend()
plt.grid(True)
plt.show()
```

Explanation

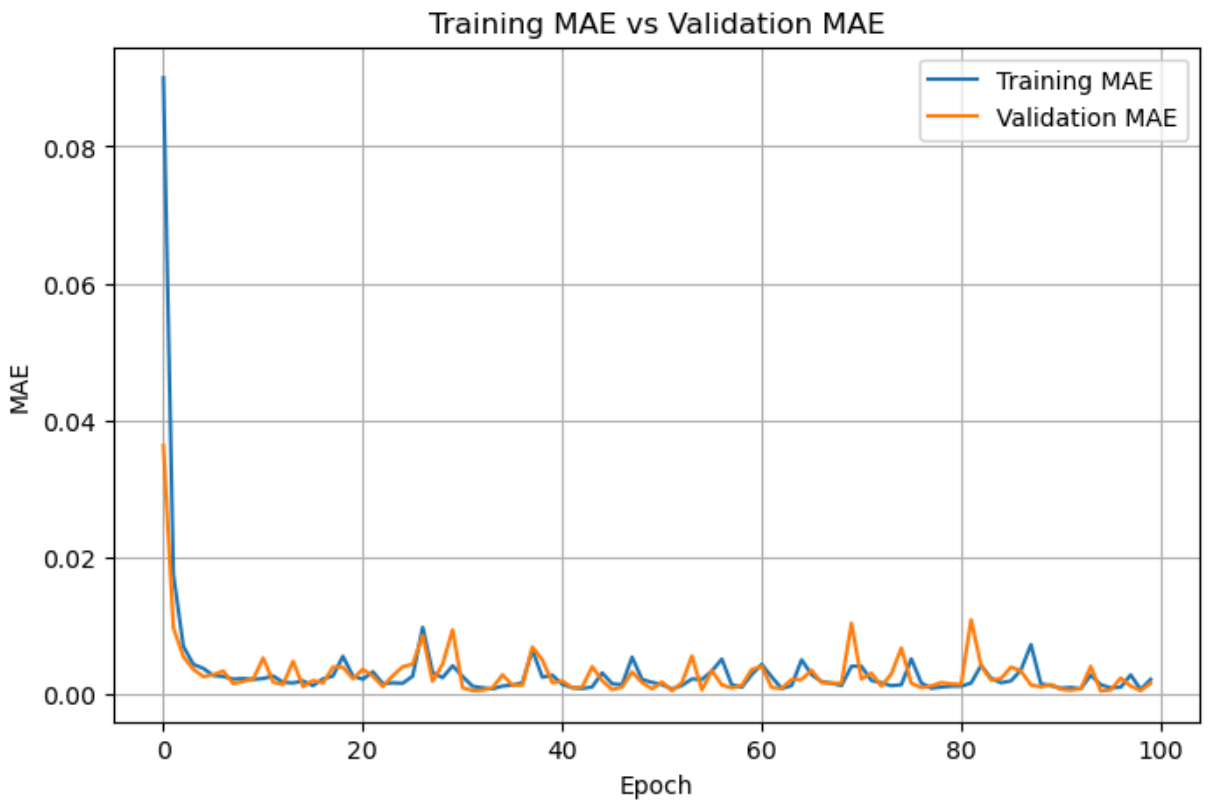
- `plt.figure(figsize=(8,5))`
  - Creates a figure with dimensions **8 × 5 inches**.
- `history.history['mae']`
  - Retrieves the **training Mean Absolute Error (MAE)** recorded after each epoch.
- `history.history['val_mae']`
  - Retrieves the **validation Mean Absolute Error (MAE)** recorded after each epoch.
- `plt.plot()`
  - Plots the training and validation MAE curves.
- `plt.title("Training MAE vs Validation MAE")`
  - Adds a descriptive title to the graph.
- `plt.xlabel("Epoch")`
  - Labels the horizontal axis with the training epochs.
- `plt.ylabel("MAE")`
  - Labels the vertical axis with the Mean Absolute Error values.
- `plt.legend()`
  - Displays a legend to distinguish between the training and validation MAE curves.
- `plt.grid(True)`
  - Adds grid lines to improve the readability of the graph.
- `plt.show()`
  - Displays the completed plot.

Purpose

- Visualize how the prediction error changes during training.
- Compare the model's performance on the training and validation datasets.
- Monitor the learning process and identify potential **underfitting** or **overfitting**.
- Verify that the model is improving its predictive accuracy as the number of training epochs increases.

```
# Plot Training vs Validation MAE

plt.figure(figsize=(8,5))
plt.plot(history.history['mae'], label='Training MAE')
plt.plot(history.history['val_mae'], label='Validation MAE'
plt.title("Training MAE vs Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.legend()
plt.grid(True)
plt.show()
```



Plotting the Training and Validation Loss

This cell plots the **training loss** and **validation loss** over all training epochs to monitor the learning progress of the Deep Neural Network (DNN). The loss values are measured using **Mean Squared Error (MSE)**, which is the loss function used during model training.

Code

```
# Plot Training vs Validation Loss

plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.grid(True)
plt.show()
```

Explanation

- **`plt.figure(figsize=(8,5))`**
  - Creates a figure with dimensions **8 × 5 inches**.
- **`history.history['loss']`**
  - Retrieves the **training Mean Squared Error (MSE)** recorded after each training epoch.
- **`history.history['val_loss']`**
  - Retrieves the **validation Mean Squared Error (MSE)** recorded after each epoch.
- **`plt.plot()`**
  - Draws the training and validation loss curves on the same graph for comparison.
- **`plt.title("Training Loss vs Validation Loss")`**
  - Adds a descriptive title to the plot.

- `plt.xlabel("Epoch")`
  - Labels the horizontal axis with the training epochs.
- `plt.ylabel("MSE Loss")`
  - Labels the vertical axis with the Mean Squared Error values.
- `plt.legend()`
  - Displays a legend to distinguish between the training and validation loss curves.
- `plt.grid(True)`
  - Adds grid lines to improve the readability of the graph.
- `plt.show()`
  - Displays the completed plot.

Purpose

- Visualize how the model's **training** and **validation loss** change throughout the training process.
- Compare the model's performance on the training and validation datasets.
- Detect signs of **underfitting** or **overfitting** by observing the gap between the two curves.
- Verify that the model is converging toward an optimal solution as the number of training epochs increases.

```
# Plot Training vs Validation Loss

plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Los
plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.legend()
plt.grid(True)
plt.show()
```



Predicting the Output for a New Input

This cell uses the trained Deep Neural Network (DNN) to predict the output for a new input value, **(x = 5)**. The input is first normalized before being passed to the model, and the predicted output is then converted back to its original scale for comparison with the actual polynomial value.

Code

```
x_new = np.array([[5]])
x_new_norm = x_scaler.transform(x_new)

prediction_norm = model.predict(x_new_norm)

prediction = y_scaler.inverse_transform(prediction_norm)

actual = 5*(5**3)-10*(5**2)-20*5+10
```

Explanation

- **`x_new = np.array([[5]])`**
  - Defines a new input sample with the value **5**.
- **`x_scaler.transform(x_new)`**
  - Normalizes the input value using the same `MinMaxScaler` that was fitted on the training data.
  - This ensures the input is in the same range (**[-1, 1]**) as the data used during training.
- **`model.predict(x_new_norm)`**
  - Uses the trained Deep Neural Network to predict the normalized output corresponding to the normalized input.
- **`y_scaler.inverse_transform(prediction_norm)`**
  - Converts the predicted normalized output back to its original scale, making it directly comparable to the true polynomial value.
- **`actual = 5*(5**3)-10*(5**2)-20*5+10`**
  - Calculates the actual output of the polynomial equation for **(x = 5)** using the mathematical expression:

[y = 5x^3 - 10x^2 - 20x + 10]

Purpose

- Demonstrate how the trained model can make predictions for new, unseen input values.
- Convert the prediction back to the original scale for meaningful interpretation.
- Compare the predicted value with the actual value of the polynomial function to assess the model's prediction accuracy.

```
x_new = np.array([[5]])
x_new_norm = x_scaler.transform(x_new)

prediction_norm = model.predict(x_new_norm)

prediction = y_scaler.inverse_transform(prediction_norm)

actual = 5*(5**3)-10*(5**2)-20*5+10

```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step
```

Displaying the Prediction Results

This cell displays the input value, the output predicted by the trained Deep Neural Network (DNN), and the actual output computed from the polynomial equation. Comparing these values provides a simple way to assess the model's prediction